Institute of Geographical Information Systems

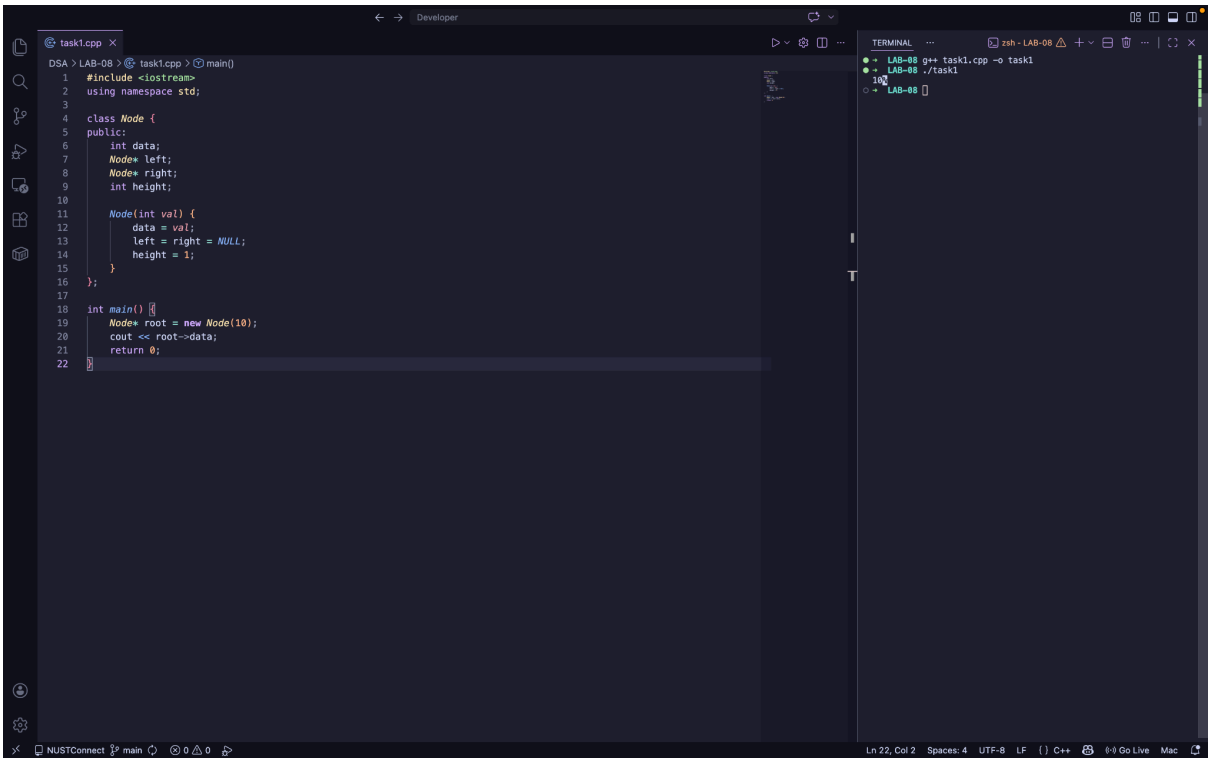
CS-250 - Data Structures and Algorithms LAB

Semester: Spring 2026	**Class: SCEE-IGIS - 2024**
Name: Ali Nawaz	**CMS ID : 00000526123**
Submitted to: Ma'am Alvina Anjum	**Due Date: April 01, 2026**

LAB 8: AVL Trees in Data Structures and Algorithms

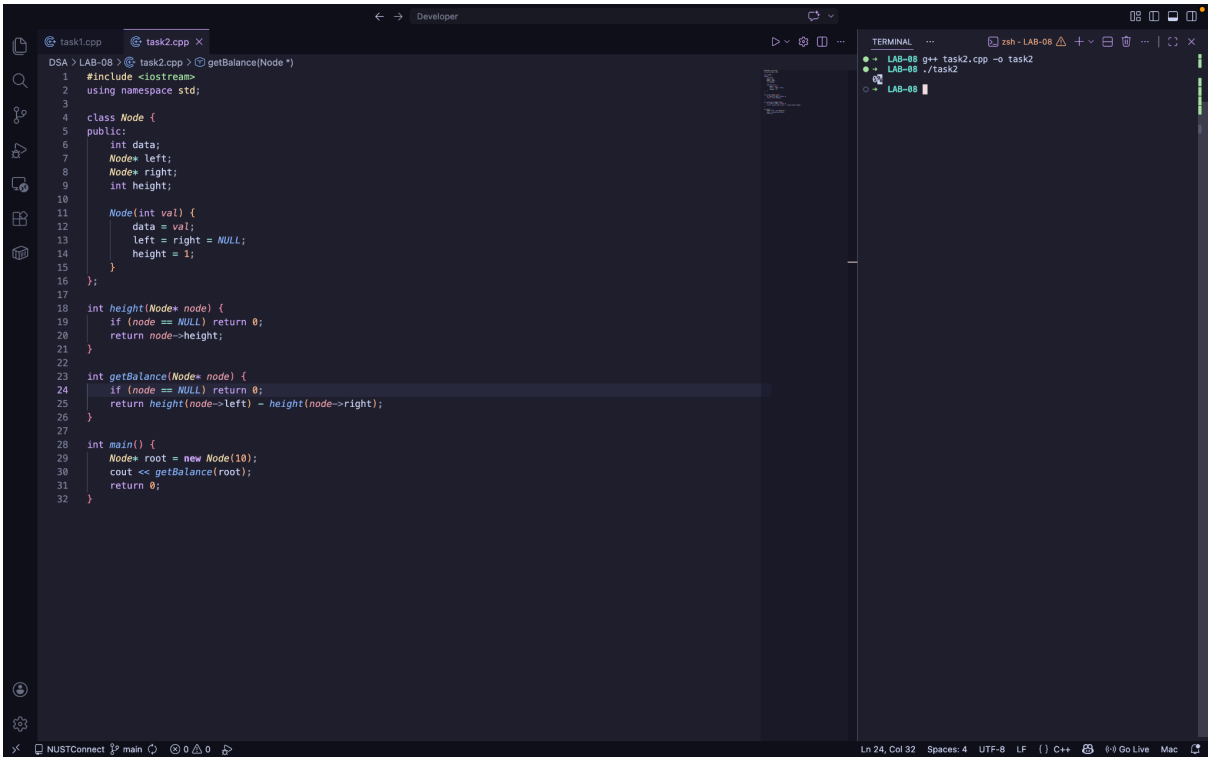
Task 1: Create a Binary Tree Structure

- Define a Node class with data, left, and right child pointers.
- Write a function to create a binary tree from user input (recursive or static).



Task 2: Height and Balance Factor Functions

- Implement a function to get the height of a node.
- Implement a function to get the balance factor of a node.



Task 3: Rotation Functions

- Implement left rotation and right rotation functions.

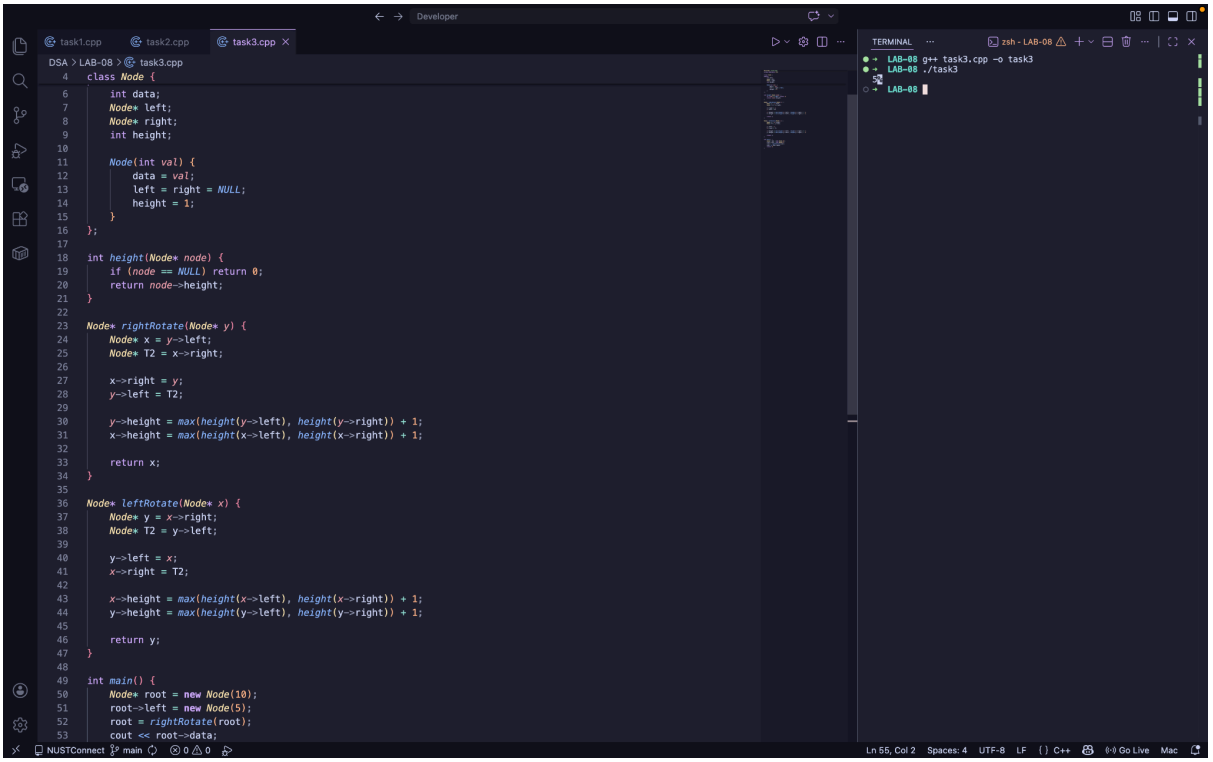
Hint: // Right rotation (single rotation)

```
// Used when a left-heavy tree needs to be balanced (left-left case)
Node* rightRotate(Node* y) {
    Node* x = y->left;          // x is the left child of y
    Node* T2 = x->right;        // T2 is the right child of x

    // Perform rotation: x becomes the new root, y becomes the right child of x
    x->right = y;
    y->left = T2;

    // Update heights of the rotated nodes
    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x; // x is the new root of the rotated subtree
}
```

- Implement left-right and right-left rotation if needed.

Hint: // Left-Right rotation (double rotation)

```
// Used when a left-heavy subtree is right-heavy (left-right case)
Node* leftRightRotate(Node* node) {
    node->left = leftRotate(node->left); // First perform a left rotation
    return rightRotate(node);            // Then perform a right rotation
}
```



Task 4: Insertion in AVL Tree

- Implement a function to insert a node while maintaining the AVL balance property.

Hint: Update height of ancestor node

node->height = 1 + max(height(node->left), height(node->right));

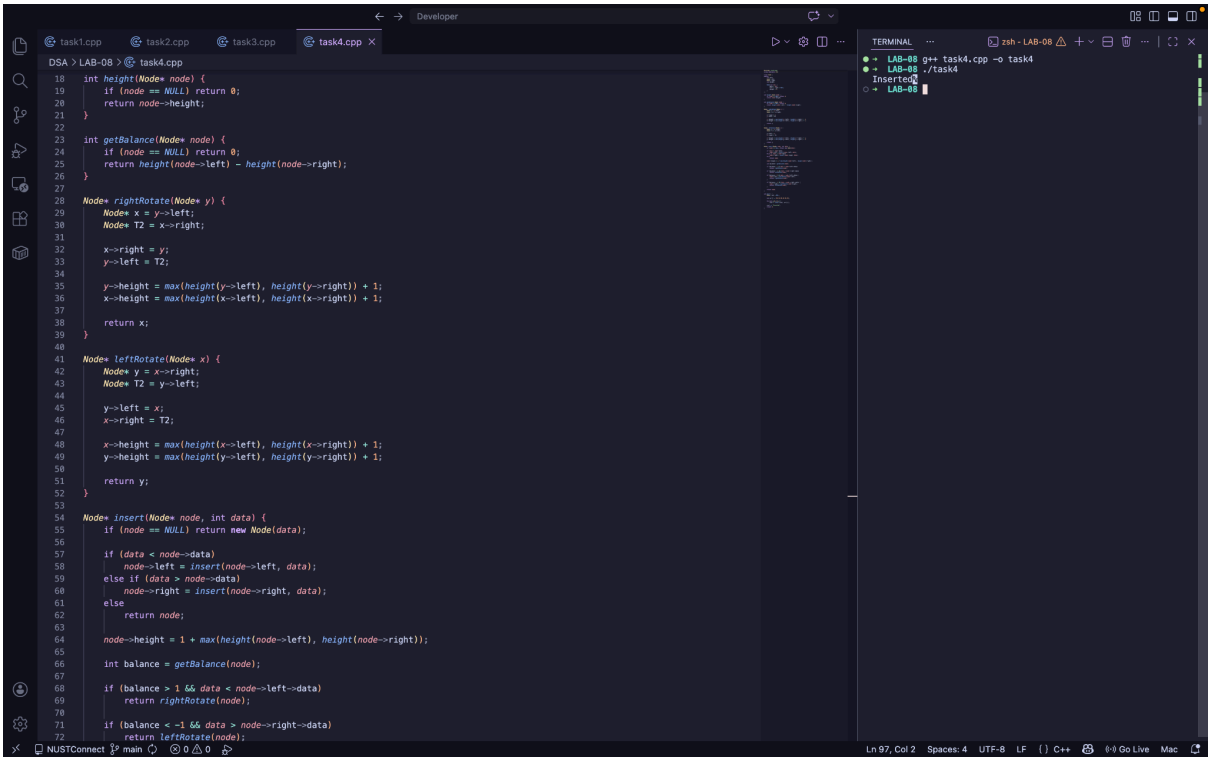
//Balance the tree (If it is unbalanced, perform rotations)

// Left Left Case (left-heavy subtree)

if (balance > 1 && data < node->left->data)

return rightRotate(node);

- Test the function by inserting the values: 10, 20, 30, 40, 50, 25.

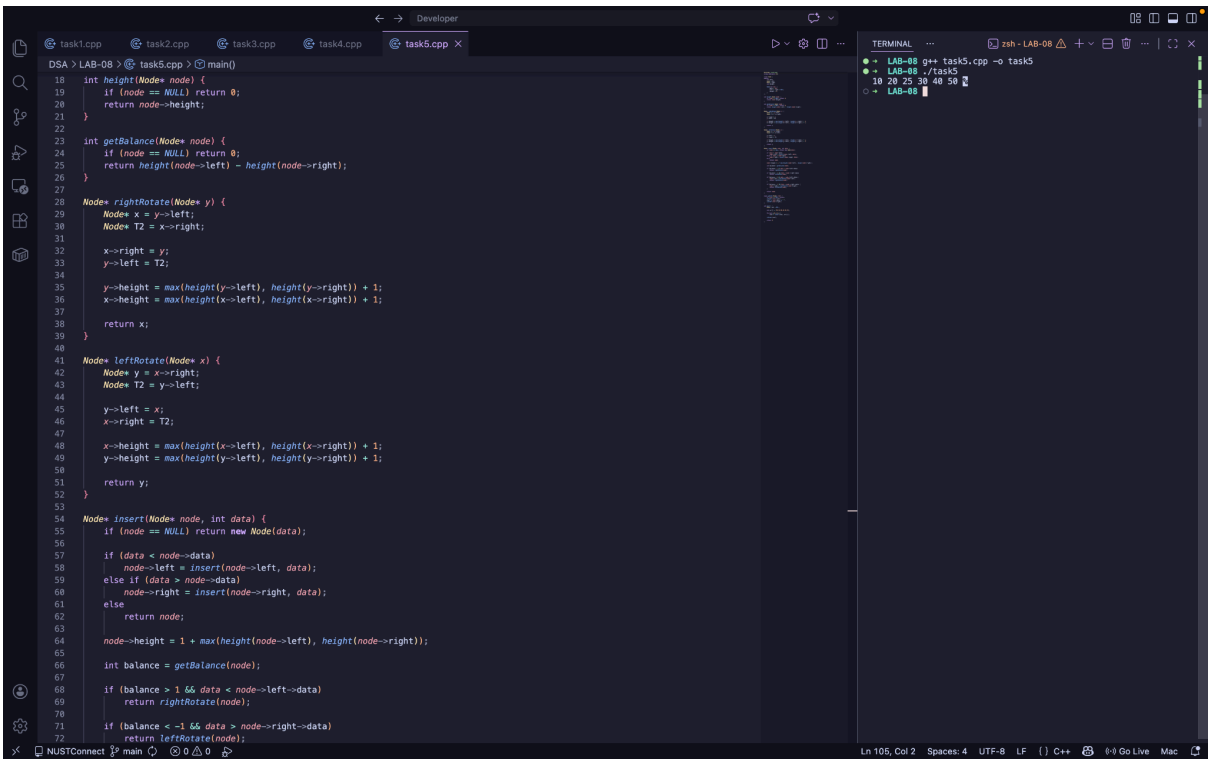


Task 5: Traversal of AVL Tree

- Write a function for in-order traversal of the AVL tree.

```
void inOrder(Node* root) {
  }
```

- Display the tree structure before and after insertions.



Report: AVL Tree Lab

In this lab, I implemented an AVL tree, which is a self-balancing binary search tree. The main goal was to understand how trees stay balanced automatically after insertion.

First, I created a node structure that stores data, left and right child pointers, and height. The height is important because it helps in checking whether the tree is balanced or not.

Then, I implemented functions to calculate the height of a node and the balance factor. The balance factor shows the difference between the heights of left and right subtrees.

After that, I implemented rotation functions such as left rotation and right rotation. These rotations are used to fix the tree when it becomes unbalanced.

Next, I implemented the insertion function. While inserting values into the tree, I updated the height of nodes and checked the balance factor. If the tree became unbalanced, I applied the correct rotation to balance it again.

Finally, I implemented inorder traversal to display the tree. This helped in verifying that the AVL tree was working correctly and remained sorted.

What I Learned

- AVL trees are self-balancing binary search trees
- Balance factor helps detect imbalance
- Rotations are used to maintain balance
- Recursion is important in tree operations
- AVL trees improve performance by keeping height balanced